Here is your complete MASTER PROMPT for Android Studio

(copy-paste this entire block into Gemini in Android Studio, Cursor, Claude, or any AI coding assistant inside your project).

It already contains **full analysis**, **exact business logic**, **flow**, **decimal fixes**, **modern attractive redesign**, **new features**, and **debug instructions**.

MASTER PROMPT (ready to paste):

You are an expert Android Kotlin developer with 10+ years experience. You specialize in beautiful, modern Jetpack Compose + Material 3 apps.

I have already built a petrol pump management app called **FuelBook** (for HP Pump). I am giving you **exact screenshots** (described below) and I want you to **completely refactor and upgrade** the entire project to the latest 2026 standards.

1. Full Analysis of Current App (from screenshots)

- App name: FuelBook (HP Pump)

- User: Vishal Chaudhri

- Current tech: Looks like old XML + Activities/Fragments (blue theme, card views, basic buttons)

- Screens:

  1. Home/Dashboard – 6 big cards: FUEL ENTRY, CASH COLLECTION, UPI COLLECTION, DAILY SUMMARY, HISTORY, SETTINGS + red LOGOUT

  2. Fuel Entry (date shown) – Petrol (₹104/L) & Diesel (₹90/L) sections. 4 nozzles each. Fields: Opening, Closing, Litres, Amount. Totals below. SAVE button.

  3. Cash Collection – Denominations (500,200,100,50,20,10) + Count + Amount + Coins Total + Total Cash. SAVE.

  4. UPI Collection – Paytm, Google Pay, PhonePe, Others + Amount + Total. SAVE.

5. Daily Summary – Shows Diesel Sales, Cash, UPI, Fuel Total (Petrol+Diesel), Grand Total (Cash+UPI). SAVE DAILY REPORT + DOWNLOAD PDF.

6. History / Filter Reports – Date range filter + list of daily reports with Petrol ₹, Diesel ₹, Grand Total. Tap opens popup with full details + Difference (red if negative) + GENERATE, DELETE, OK.

7. Settings – Edit Fuel Prices, Manage Nozzles (add/remove N1-N4), UPI Options (add/remove), red LOGOUT.

Current problems:

- Many numeric fields do NOT accept decimal values (litres must be 12.75, amounts 1234.50, etc.).

- No live auto-calculation (Closing – Opening = Litres, Litres × Price = Amount).

- UI is outdated, boring blue, not colorful or premium.

- Dates inconsistent in screenshots (use proper DatePicker).

- No validation, no animations, no charts.

2. Exact Business Logic & Core Flow (MUST IMPLEMENT/CORRECT)

Daily workflow (one date = one report):

1. FUEL ENTRY (per date)

   - Two sections: Petrol & Diesel

   - 4 nozzles (N1-N4) – configurable in settings

   - For each nozzle: Opening (decimal), Closing (decimal)

   - Litres = max(0, Closing - Opening) → auto-calculate live

   - Amount = Litres × Current Price → auto-calculate live

   - Total Petrol Litres & Amount, Total Diesel Litres & Amount

   - All fields MUST support 2 decimal places (e.g. 123.75 L, ₹1234.50)

2. CASH COLLECTION (same date)

- Fixed denominations: 500,200,100,50,20,10 + Coins
- Count (integer) → Amount = Count × Denom (auto)
- Total Cash (decimal allowed for paise if needed)

3. UPI COLLECTION (same date)
- Paytm, Google Pay, PhonePe, Others (configurable)
- Amount per method (decimal)
- UPI Total

4. DAILY SUMMARY (auto-generated)
- Diesel Sales (litres + ₹)
- Petrol Sales (litres + ₹) – must show both
- Cash Collection ₹
- UPI Collection ₹
- Fuel Total = Petrol ₹ + Diesel ₹
- Grand Total = Cash + UPI
- Difference = Fuel Total – Grand Total (green if zero, red if negative)
- Buttons: SAVE DAILY REPORT + DOWNLOAD PDF (beautiful formatted PDF with logo, date, all details)

Data persistence: Room Database (MVVM + Repository)

- Table: DailyReport (date as primary key, all above values stored)
- Table: FuelPrices (single row, petrolPrice, dieselPrice)
- Nozzles & UPI methods also stored and editable

History:

- Filter by Start Date & End Date
- List shows: Date | Petrol ₹ | Diesel ₹ | Grand Total

- On click → Detail popup with full breakdown + Difference + DELETE button

3. Design Requirements – Make it PREMIUM, COLORFUL & LATEST

- Use **Jetpack Compose + Material 3** (MaterialTheme with dynamic colors + custom Fuel theme)

- Color scheme: Vibrant modern – Deep Blue (#0A2540), Fuel Green (#00C853), Orange accents (#FF9800), White/Off-white cards, gradients

- Beautiful cards with elevation, rounded corners (16dp), subtle shadows, hover/click scale animation

- Use Material Icons + Lottie animations (fuel pump filling, money flying, etc.)

- Bottom navigation bar (Home, Entry, Reports, Settings)

- Dark mode support (auto + manual switch)

- All screens must feel like 2026 apps (Figma-level polish)

- Live calculations with debounce

- All numeric inputs: keyboardType = KeyboardType.Decimal, allow 2 decimal places

- Use DecimalFormat("0.00") everywhere for display

4. New Features to Add (must include)

1. Live auto-calculation on every field change (Fuel Entry)

2. Dashboard home with Today's quick stats + mini chart (bar for Petrol vs Diesel)

3. Charts in Daily Summary & History (Compose Charts or MPAndroidChart):

   - Pie: Cash vs UPI

   - Bar: Petrol vs Diesel sales last 7 days

4. Export options: PDF + CSV + Share via WhatsApp

5. Search in History + Sort by date/amount

6. Validation: Closing ≥ Opening, no negative values, show Snackbars

7. Settings:

   - Theme switcher (Light/Dark)

   - Add/Remove nozzles & UPI methods with swipe-to-delete

- Backup & Restore (to Google Drive or local)

8. Offline-first (Room + WorkManager sync if you add cloud later)

9. Splash screen with FuelBook logo animation

5. Technical Stack (MUST use)

- Kotlin 100%

- Jetpack Compose + Material 3

- Navigation Compose + Single Activity

- MVVM + ViewModel + StateFlow

- Room Database + Repository

- Hilt for DI

- Coroutines + Flow

- Coil for images

- iTextPDF or Compose PDF for beautiful reports

- Accompanist or official permissions

- Target SDK 36, min SDK 26

6. Instructions for You (the AI)

1. First, create a fresh Compose project structure (or refactor existing).

2. Implement EVERY screen exactly as described with premium UI.

3. Fix ALL decimal issues (use TextField with decimal keyboard + format to 2 places).

4. Implement exact business logic and calculations.

5. Add all new features listed.

6. Write clean, commented code.

7. After generating, run full debug simulation:

   - Test with sample data (400.75 L petrol, 125.50 L diesel, cash 8700, UPI 408)

   - Verify Difference shows correctly

   - Verify PDF looks professional

- Verify decimals work everywhere

8. Output the code in proper file structure (MainActivity.kt, screens/, data/, ui/theme/, etc.) with instructions on how to replace files.

Start by saying "**FUELBOOK MASTER REFACTOR STARTED**" and then generate the complete modern app step-by-step.

How to use this prompt:

1. Open your existing FuelBook project in Android Studio.

2. Paste the entire block above into Gemini (or Cursor/Claude).

3. Let it generate.

4. After it finishes, run the app and test with decimals (e.g., 123.75 litres).

This master prompt already includes **everything** you asked for: full analysis, correct business logic, decimal fix, premium colorful modern design, new features, and self-debug instructions.

If you want me to add anything extra (e.g., cloud sync with Firebase, multi-user login, barcode scanner for UPI, etc.) just say the word and I'll update the master prompt instantly.

Ready to make your FuelBook look like a ₹5 crore startup app? 🚀